

Specialization: *Data Engineering*

Business Focus: *SQL Database Design & Query Optimization*

Tool: *PostgreSQL*

SwiftRide Logistics Database Management System

Project Learning Opportunities

This project exposes students to practical database engineering and relational database management using PostgreSQL

Learning Objectives

Students will learn how to:

- Design relational databases
- Create tables using DDL statements
- Insert, update, and delete records using DML
- Query data using DQL
- Work with: JOINS

Implement:

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- CHECK
- NOT NULL
- DEFAULT

Use:

- ON DELETE CASCADE
- ON UPDATE CASCADE

Perform:

- Filtering
- Sorting
- Aggregations
- Grouping
- Subqueries

Understand data relationships:

- One-to-One
- One-to-Many
- Many-to-Many



Case Study Overview

About the Organization

SwiftRide Logistics is a fast-growing logistics and delivery company operating across major cities in Nigeria.

The company handles:

- Customer orders**
- Delivery operations**
- Driver assignments**
- Warehouse management**
- Vehicle tracking**
- Payments and invoices**

As the business expanded, the company realized its manual record-keeping system could no longer support growing operational demands.

The management decided to build a centralized PostgreSQL database system to automate operations and improve reporting.



Case Study Overview

Problem Statement

SwiftRide Logistics currently faces several business challenges:

- Duplicate customer records**
- Delayed delivery tracking**
- Missing payment information**
- Poor relationship management between orders and drivers**
- Inconsistent warehouse inventory records**
- Slow reporting and operational inefficiencies**
- Difficulty managing deleted records and dependencies**

Additionally:

- Drivers sometimes get assigned to invalid orders**
- Orders remain even after customers are removed**
- Inventory records become inconsistent after updates**

The company needs a robust relational database system with proper constraints, relationships, and cascading rules to maintain data integrity and improve business operations.



Proposed Solution



1.

Build a PostgreSQL-powered relational database management system that:

- ❖ **Centralizes operational data**
- ❖ **Uses normalized relational tables**
- ❖ **Enforces constraints and validations**
- ❖ **Implements cascading operations**
- ❖ **Supports efficient querying and reporting**
- ❖ **Provides reliable relationships between entities**

2.

The system will include modules for:

- Customers
- Orders
- Drivers
- Deliveries
- Warehouses
- Inventory
- Payments
- Vehicles



Rationale for the Project

(What is the Importance of the project to the business)

1.

Improved Data Integrity

Constraints ensure that invalid or duplicate data cannot enter the database.

2.

Better Relationship Management

Foreign keys and joins help connect related business data efficiently.

3.

Automated Dependency Management

Cascade operations automatically manage related records during updates or deletions.

4.

Faster Reporting

DQL queries and joins enable quick generation of operational reports.

5.

Real-World SQL Experience

Students gain hands-on exposure to enterprise-level relational database design.



Database Schema

Architectural Overview

The SwiftRide database follows a relational database architecture where all operational data is connected through primary and foreign key relationships.

Core Relationships

One-to-Many

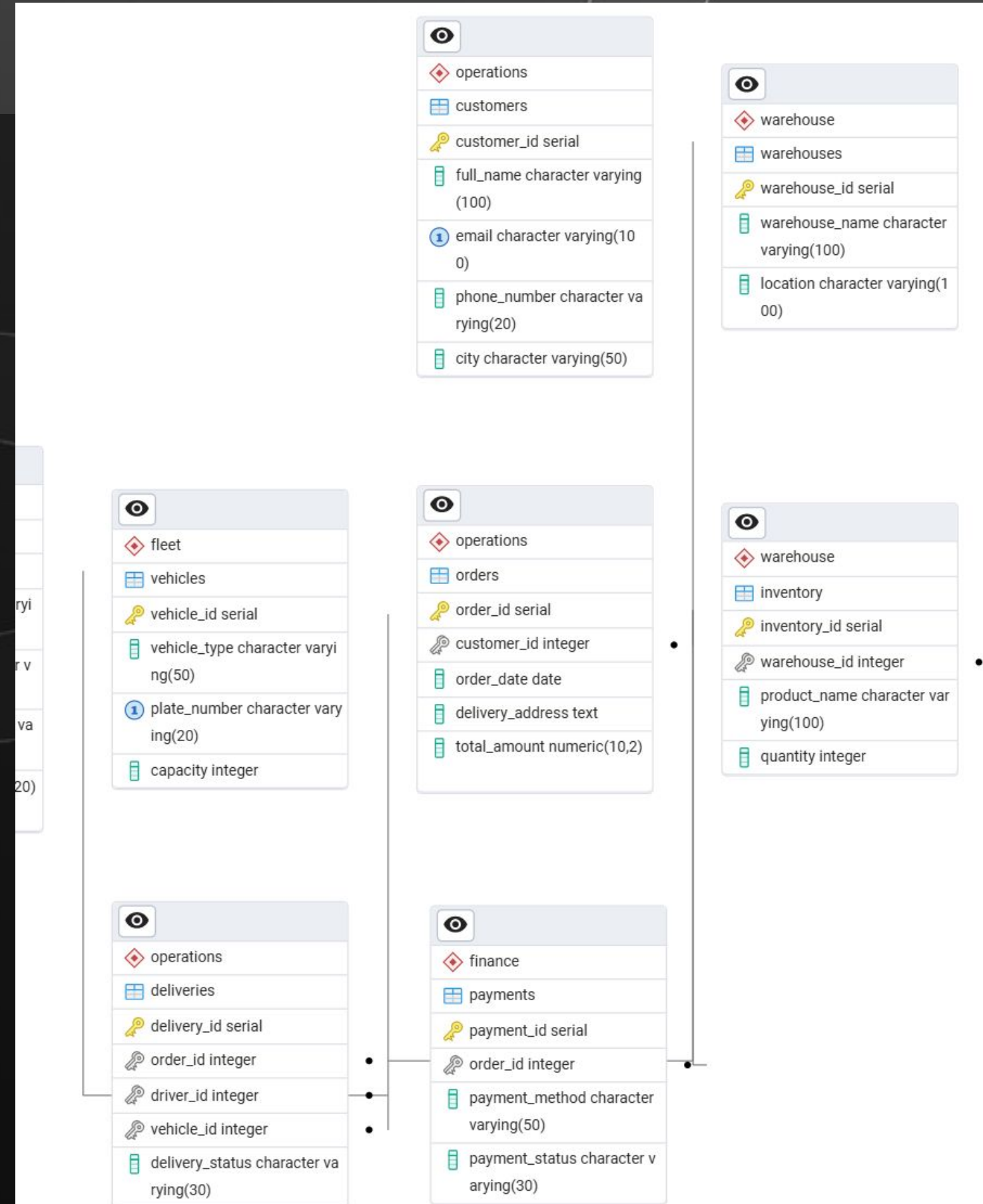
- One customer → Many orders
- One driver → Many deliveries
- One warehouse → Many inventory records

Many-to-One

- Many orders → One customer
- Many deliveries → One vehicle

Referential Integrity

Foreign keys enforce valid relationships between tables.



Database Schema



Operations Schema

The operations schema manages the core business activities and logistics transactions of SwiftRide Logistics.

- Stores operational business tables such as customers, orders, and deliveries.
- Handles customer order processing and delivery tracking activities.
- Maintains relationships between customers, orders, drivers, and vehicles using foreign keys.
- Supports day-to-day logistics operations and delivery workflow management.
- Demonstrates enterprise concepts like joins, cascade operations, and referential integrity.

Database Schema



Fleet Schema

The fleet schema manages transportation resources used for deliveries.

- Stores fleet-related tables such as drivers and vehicles.
- Tracks driver information, including license numbers, contact details, and work status.
- Maintains vehicle records such as plate numbers, vehicle types, and carrying capacities.
- Helps monitor delivery assignments and fleet utilization across the logistics system.
- Demonstrates the use of constraints, validations, and one-to-many relationships in PostgreSQL.

Database Schema



Warehouse Schema

The warehouse schema handles inventory and warehouse management operations.

- Stores warehouse- and inventory-related tables, such as warehouses and inventory.
- Tracks product quantities and stock availability across multiple warehouse locations.
- Supports inventory monitoring and low-stock analysis for business decision-making.
- Maintains relationships between warehouses and inventory items using foreign keys.
- Demonstrates inventory management concepts and relational database organization.

Database Schema



Finance Schema

The finance schema manages payment and financial transaction records.

- Stores the payments table containing payment methods and payment statuses.
- Tracks successful, pending, failed, and refunded payment transactions.
- Helps monitor company revenue and customer payment activities.
- Maintains relationships between payments and customer orders through foreign keys.
- Demonstrates financial reporting, payment analysis, and transaction integrity in SQL systems.

**READY TO
DELVE IN?**

